

BetinAsia Scrapping - Descobertas e Documentacao

Data: Janeiro/Fevereiro 2026

Projeto: BetinAsia Bot - Coleta de Odds

Status: Producao

1. Visao Geral do Site

1.1 Estrutura de URLs

BASE_URL: `https://black.betinasia.com`

Paginas principais:

- `/sportsbook/football` -> Todos os jogos de futebol
- `/sportsbook/football/XE/1` -> England Premier League
- `/sportsbook/football/XE/1/2026-02-01,2,12` -> Jogo especifico

Formato do Event ID: `YYYY-MM-DD,home_id,away_id`

Exemplo: `2026-02-01,2,12` (Aston Villa vs Brentford)

1.2 Autenticacao

- Login via formulario web (usuario/senha)
- Sessao mantida via cookies (22 cookies diferentes)
- Token de sessao: header `session` nas requisicoes API
- Sessao pode ser salva/restaurada via Playwright

2. Arquitetura de Dados do Site

2.1 WebSocket (Fonte Principal de Dados)

URL: `wss://black.betinasia.com/cpricefeed/?token={session_token}&lang=en`

O site usa WebSocket para streaming de dados em tempo real. Esta e a fonte mais importante e eficiente para coleta de odds.

Tipos de Mensagens WebSocket:

Tipo	Descricao	Conteudo
event	Info dos jogos	Times, liga, horario, pais
offers_hcap	Odds agregadas (AH, OU, 1X2)	Best odds de todos bookmakers
offers_event	Similar a offers_hcap	Odds adicionais
synced	Sincronizacao	Balance, exchange rates
pong	Keep-alive	Timestamp
error	Erros	event_already_subscribed, etc

Exemplo de Mensagem event:

```
[["event",["fb","2026-02-01,2,12"],{
  "event_type": "normal",
  "start_ts": "2026-02-01T14:00:00Z",
  "competition_id": 1,
```

```

    "competition_name": "England Premier League",
    "country": "XE",
    "home": "Aston Villa",
    "away": "Brentford",
    "event_name": "Aston Villa vs. Brentford"
  ]
}
```

Exemplo de Mensagem offers_hcap:

```

[["offers_hcap",[1,"fb","2026-02-01,2,12"],{
  "ah": [-1, [{"a", 2.1}, {"h", 1.819}]],
  "ahou": [2.5, [{"over", 1.847}, {"under", 2.05}]],
  "wdw": [null, [{"a", 3.67}, {"d", 3.49}, {"h", 2.1}]]
}]]

```

2.2 APIs REST

APIs Publicas (sem rate limit agressivo):

Endpoint	Metodo	Descricao
/api/version	GET	Versao do sistema
/api/ping	GET	Health check
/v1/customers/{user}/accounting_info/	GET	Balance, P/L
/v1/customers/{user}/bookie_accounts/	GET	Contas de bookmakers
/v1/events/{user}/suggested/	GET	Eventos sugeridos
/v1/competitions/recommended/	GET	Ligas recomendadas
/v1/orders/	GET	Historico de apostas

API de Betslip (RATE LIMITED!):

Endpoint	Metodo	Descricao	Rate Limit
/v1/betslips/	POST	Abre betslip para uma aposta	1 por minuto

IMPORTANTE: A API de betslips tem rate limit severo. Apos exceder:

```

{
  "status": "error",
  "code": "throttled",
  "data": {
    "message": "Request was throttled. Expected available in 4855 seconds.",
    "retry_after": 4855
  }
}

```

Isso significa ~**81 minutos** de bloqueio!

2.3 WebSocket PMM (Price Market Maker)

Apos abrir um betslip, o site envia odds detalhadas por bookmaker via WebSocket:

```

[["api",{ "ts":1769907872.06,"data":[
  ["pmm",{

```

```

    "betslip_id": "ccd473ebf09b43d488b95b6e71d69be0",
    "sport": "fb",
    "event_id": "2026-02-01,2,12",
    "bookie": "pin88",
    "username": "_a26392e3_",
    "bet_type": "for,h",
    "status": {"code": "success"},
    "price_list": [
      {"effective": {"price": 2.1, "min": ["GBP", 0.8657], "max": ["GBP", 5477.655]}}
    ]
  }
}]
]]]]

```

Bookmakers identificados: pin88, sbo, sharp, bf, bdaq, mbook, 4casters, sing2, sxbet, overtime, ibc

3. Estrategias de Scraping

3.1 Metodo LENTO (Cliques - NAO RECOMENDADO)

Problema: Cada clique em uma odds abre um betslip e conta no rate limit.

Fluxo:

1. Navega para pagina do jogo
2. Clica em cada odds para abrir painel de bookmakers
3. Extraí bookmakers do DOM
4. Fecha painel (ESC)
5. Repete para cada linha de handicap

Resultado: ~10-20 cliques por jogo = rate limit em segundos

3.2 Metodo RAPIDO (WebSocket - RECOMENDADO)

Solucao: Capturar dados do WebSocket sem interacao.

Fluxo:

1. Navega para pagina de futebol OU liga
2. Aguarda 6 segundos
3. Parseia mensagens WebSocket
4. Extraí offers_hcap e offers_event

Resultado: ~250 jogos em 10 segundos, sem rate limit

3.3 Comparacao de Metodos

Aspecto	Cliques	WebSocket
Velocidade	~6 min/jogo	~10s total
Rate limit	Sim (1/min)	Nao
Dados	Odds por bookmaker	Best odds agregadas
Jogos/ciclo	~10 antes de bloqueio	~250
Complexidade	Alta	Baixa

4. Cobertura de Dados

4.1 Navegacao para /sportsbook/football

Uma unica navegacao para a pagina principal de futebol retorna:

- ~800-1000 eventos de futebol (metadados)
- ~250 eventos com odds (offers_hcap)
- ~19 ligas com odds automaticas

4.2 Ligas com Odds Automaticas

England Premier League: 21-26 jogos

England Championship: 15-16 jogos

England League 1: 15 jogos

England League 2: 16 jogos

England National League: 8-11 jogos

Spain La Liga: 22 jogos

Germany Bundesliga: 18 jogos

Italy Serie A: 22 jogos

France Ligue 1: 23 jogos

Portugal Primeira Liga: 8 jogos

Scotland Premier League: 6 jogos

UEFA Champions League: 12 jogos

UEFA Europa League: 8 jogos

FIFA World Cup: 27 jogos

Brazil (varios): 1-3 jogos

4.3 Ligas que Requerem Navegacao Especifica

Ligas menores (ex: Spain Tercera Division com 50 jogos) nao recebem odds automaticamente na pagina principal. Para estas, e necessario navegar especificamente para a pagina da liga.

5. Estrutura de Dados Coletados

5.1 Asian Handicap (AH)

```
{
  "ah_lines": {
    0.0: {"home_odds": 1.70, "away_odds": 2.39},
    -0.25: {"home_odds": 1.55, "away_odds": 2.58},
    -0.5: {"home_odds": 1.93, "away_odds": 2.03},
    -0.75: {"home_odds": 2.10, "away_odds": 1.85},
    -1.0: {"home_odds": 2.50, "away_odds": 1.62},
    # ... ate ~21 linhas
  }
}
```

5.2 Over/Under (OU)

```
{
  "over_under": {
    2.5: {"over": 1.85, "under": 2.05},
    3.0: {"over": 2.10, "under": 1.80},
    # ...
  }
}
```

```
}  
}
```

5.3 Match Odds (1X2)

```
{  
  "match_odds": {  
    "h": 2.10,  # Home  
    "d": 3.49,  # Draw  
    "a": 3.67   # Away  
  }  
}
```

6. Codigo de Implementacao

6.1 FastCollector (Recomendado)

Arquivo: /betinasia_bot/scrapper/fast_collector.py

```
from scraper.fast_collector import FastCollector  
  
async def main():  
    collector = FastCollector()  
    await collector.start()  
  
    # Coleta TUDO em uma navegacao  
    result = await collector.collect_all()  
  
    print(f"Coletados {result.total_with_odds} jogos")  
    print(f"Ligas: {result.leagues_with_odds}")  
  
    for match in result.matches:  
        print(f"{match.home_team} vs {match.away_team}")  
        for line, odds in match.ah_lines.items():  
            print(f"  AH {line:+.2f}: H={odds.home_odds} A={odds.away_odds}")  
  
    await collector.close()
```

6.2 WebSocketCollector (Por Liga)

Arquivo: /betinasia_bot/scrapper/websocket_collector.py

```
from scraper.websocket_collector import WebSocketCollector  
  
async def main():  
    collector = WebSocketCollector()  
    await collector.start()  
  
    # Coleta uma liga especifica  
    matches = await collector.collect_league("England Premier League")  
  
    for match in matches:  
        print(f"{match.home_team} vs {match.away_team}")  
  
    await collector.close()
```

7. Headers Necessarios para API

Para chamar APIs REST diretamente (sem Playwright):

```
headers = {  
    "session": "60ebd17b896798a7a9f1c4be8fcf5489", # Token da sessao  
    "x-molly-client-name": "sonic",  
    "x-molly-client-version": "2.5.20",  
    "accept": "application/json",  
    "content-type": "application/json",  
    "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"  
}
```

8. Limitacoes e Consideracoes

8.1 Rate Limits

Recurso	Limite	Consequencia
Betslip API	1/min	Bloqueio de ~81 minutos
WebSocket	Ilimitado	Nenhuma
Navegacao	~razoavel	Captcha eventual

8.2 Dados NAO Disponiveis via WebSocket

- Odds **por bookmaker individual** (apenas best odds agregadas)
- Para odds detalhadas por bookmaker, seria necessario abrir betslips (rate limited)

8.3 Sessao

- Sessao expira apos periodo de inatividade
 - Recomendado validar sessao antes de cada ciclo de coleta
 - Cookies podem ser salvos/restaurados
-

9. Arquivos do Projeto

```
betinasia_bot/  
  scraper/  
    betinasia.py      # Scraper base com Playwright  
    fast_collector.py # Coletor rapido (WebSocket)  
    websocket_collector.py # Coletor por liga  
    api_client.py     # Cliente API REST  
  storage/  
    database.py       # PostgreSQL  
    models.py         # SQLAlchemy models  
  config/  
    settings.py       # Configuracoes  
  main.py             # Entry point
```

10. Proximos Passos Sugeridos

1. **Integracao com banco de dados:** Salvar dados coletados pelo FastCollector
 2. **Scheduler:** Executar coleta a cada X minutos
 3. **Monitoramento:** Alertas para sessao expirada ou rate limit
 4. **Ligas adicionais:** Implementar coleta de ligas menores sob demanda
-

11. Contato e Manutencao

Repositorio: <https://github.com/GuilhermeAfonsoBet/Bets>

Branch: cursor/opera-o-apostas-betinasia-93f8

Documento gerado em 01/02/2026